



RELATÓRIO TÉCNICO

Projeto Final: Cofre Digital com Senha Programável

ALUNOS

Aluno(a): Tiago Navarro Bettanin

R.A: 2477866

Aluno(a): João Antonio Sitta Martins

R.A: 2553058

1 Introdução

O projeto final desenvolvido consiste em um **sistema de cofre digital com senha programável**, implementado em VHDL para a placa DE10-Lite. A proposta do circuito é simular o funcionamento de um cofre eletrônico simples: ao ligar a placa, o sistema inicia fechado, permite a entrada de uma senha pelas chaves, verifica a senha digitada, libera a abertura em caso de acerto e bloqueia novas tentativas após cinco erros consecutivos.

A escolha deste tema foi feita por permitir a aplicação de vários conceitos trabalhados durante a disciplina de Lógica Reconfigurável, como processos sequenciais, registradores, contadores, uso de *package*, funções, displays de sete segmentos e máquinas de estados finitos. Além disso, o projeto possui comportamento interativo na placa, pois utiliza chaves, botões, LEDs e displays para representar o funcionamento do sistema.

A placa DE10-Lite possui os recursos necessários para esta implementação, incluindo clock de 50 MHz, 10 chaves deslizantes, 2 botões, 10 LEDs e 6 displays de sete segmentos. Os displays são de anodo comum, ou seja, cada segmento é acionado com nível lógico baixo. Essa característica foi considerada na definição dos padrões de acendimento utilizados no código VHDL [2].

2 Objetivo

O objetivo geral do projeto é implementar um cofre digital em VHDL, utilizando a placa DE10-Lite, capaz de verificar uma senha, indicar abertura ou fechamento por meio dos displays e bloquear o sistema após um número máximo de tentativas incorretas.

De forma mais específica, o projeto busca atender aos seguintes pontos:

- iniciar o sistema no estado fechado, exibindo a mensagem CLOSE;
- permitir a entrada de uma senha de quatro posições utilizando as chaves SW[8:0];

- confirmar cada posição da senha por meio do botão KEY0;
- exibir OPEN quando a senha estiver correta;
- exibir FAIL quando a senha estiver incorreta;
- contar a quantidade de erros consecutivos;
- bloquear o sistema após cinco tentativas incorretas;
- exibir LOCK e um contador regressivo configurado pelo genérico LOCK_TIME durante o bloqueio;
- permitir a redefinição da senha quando o cofre estiver aberto;
- organizar o projeto utilizando um arquivo principal e um *package* auxiliar.

3 Descrição Geral do Funcionamento

Ao ligar a placa, o sistema inicia no estado ST_CLOSE. Neste estado, os displays exibem a mensagem CLOSE, indicando que o cofre está fechado. Para tentar abrir o cofre, o usuário ajusta uma combinação nas chaves SW[8:0] e pressiona KEY0. Cada pressionamento de KEY0 confirma uma posição da senha.

A senha possui quatro posições. Cada posição não é apenas um dígito decimal simples, mas sim uma combinação binária formada pelas nove chaves SW0 até SW8. Dessa forma, cada posição da senha pode assumir valores de 0 até 511. A senha inicial definida no código é (1, 2, 3, 4). Como a entrada é feita por SW[8:0], estes valores correspondem às combinações apresentadas na Tabela 1.

Tabela 1: Combinações necessárias para a senha inicial.

Posição	Valor	Chaves ligadas
1 ^a posição	1	SW0
2 ^a posição	2	SW1
3 ^a posição	3	SW1 e SW0
4 ^a posição	4	SW2

Após a confirmação da quarta posição, o circuito compara a senha digitada com a senha armazenada. Se a senha estiver correta, o sistema passa para o estado ST_OPEN, acende o LED de cofre aberto e mostra OPEN nos displays. Se a senha estiver incorreta, o estado ST_FAIL é acionado, a mensagem FAIL é exibida e o contador de erros é incrementado.

Quando o número de erros chega a cinco, o sistema entra no estado ST_LOCK. Nesse estado, as tentativas são ignoradas, os LEDs piscam e os displays mostram LOCK junto com o contador regressivo configurado pelo genérico LOCK_TIME. Após o fim da contagem, o contador de erros é zerado e o sistema retorna automaticamente para CLOSE.

4 Entradas e Saídas Utilizadas

A Tabela 2 resume os sinais utilizados na entidade principal do projeto.

Tabela 2: Entradas e saídas utilizadas no projeto.

Sinal	Função no projeto
CLOCK_50	Clock principal da placa, com frequência de 50 MHz.
SW[8:0]	Combinação utilizada para formar cada posição da senha.
SW[9]	Reset geral do sistema, mantendo o comportamento do código original. Quando ativado, retorna para CLOSE, restaura a senha inicial e zera os erros.
KEY[0]	Botão de confirmação. Confirma cada posição da senha, fecha o cofre quando está aberto e confirma cada posição da nova senha no modo de redefinição.
KEY[1]	Cancela a digitação atual. Quando o cofre está aberto, entra no modo de redefinição de senha.
LEDR[0]	Indica que o cofre está aberto.
LEDR[1]	Indica que o cofre está fechado ou em processo de verificação.
LEDR[2]	Indica erro de senha.
LEDR[3]	Indica o modo de redefinição de senha.
LEDR[8:4]	Indicam a quantidade de erros acumulados.
LEDR[9:0]	Piscam durante o estado de bloqueio.
HEX5 a HEX0	Displays de sete segmentos usados para mostrar CLOSE, PASS, OPEN, SET, FAIL, LOCK e o tempo de bloqueio.

5 Máquina de Estados

O comportamento principal do cofre foi organizado por meio de uma máquina de estados finitos. Essa abordagem facilita a separação entre os modos de funcionamento do sistema, como entrada de senha, verificação, abertura, falha e bloqueio.

Tabela 3: Estados utilizados no projeto.

Estado	Descrição
ST_CLOSE	Estado inicial. Mostra CLOSE e aguarda o primeiro valor da senha.
ST_INPUT	Recebe as posições restantes da senha, uma por vez, após pressionar KEY0.
ST_VERIFY	Compara a senha digitada com a senha armazenada.
ST_OPEN	Indica cofre aberto, mostra OPEN e permite redefinir a senha.
ST_SET	Recebe uma nova senha de quatro posições.
ST_FAIL	Indica senha incorreta e incrementa o contador de erros.
ST_LOCK	Bloqueia o sistema pelo tempo definido em LOCK_TIME após cinco erros.

A Figura 1 apresenta uma representação simplificada da máquina de estados utilizada.

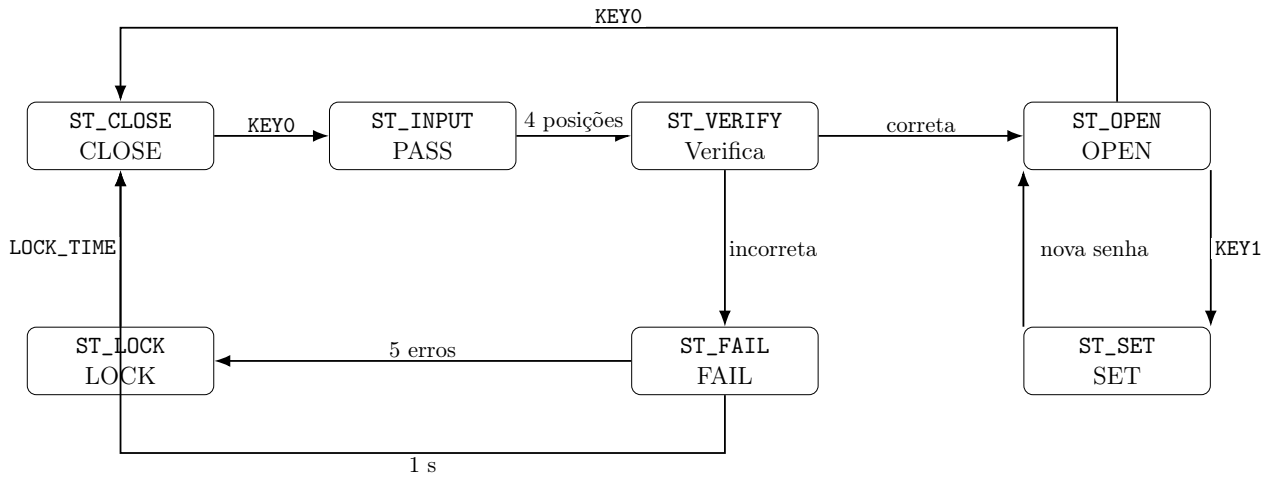


Figura 1: Representação simplificada da máquina de estados do cofre digital.

6 Temporização e Bloqueio

Para implementar o bloqueio temporizado, foi utilizado o clock principal da placa, definido pelo genérico `CLK_FREQ`. Como o valor padrão é 50000000, o código conta 50 milhões de ciclos para gerar um pulso de um segundo, chamado `tick_1hz`. Esse mesmo processo também alterna o sinal `blink`, usado para fazer os LEDs piscarem no estado de bloqueio.

Durante o estado `ST_LOCK`, o sinal `lock_seconds` é decrementado a cada pulso de 1 Hz. Enquanto a contagem não termina, o sistema ignora novas entradas de senha. Quando o contador chega a zero, o estado retorna para `ST_CLOSE` e o contador de erros é zerado.

Além disso, os botões foram amostrados em uma frequência menor, definida pelo genérico `KEY_SAMPLE_HZ`. No código, esse valor foi definido como 20 Hz, reduzindo o efeito de repiques mecânicos simples e permitindo detectar a transição de botão solto para botão pressionado.

No código apresentado no Apêndice B, o genérico `LOCK_TIME` foi mantido com valor 30, conforme o funcionamento originalmente planejado para o projeto. Durante a apresentação ou testes rápidos em bancada, esse valor pode ser reduzido temporariamente, mas a versão final documentada considera bloqueio de 30 segundos.

7 Arquivo Package

O projeto foi dividido em dois arquivos VHDL. O arquivo `cofre_pkg.vhd` contém constantes e funções auxiliares para os displays de sete segmentos. Nele são definidos os padrões das letras utilizadas nas mensagens do cofre, como C, L, O, S, E, F, A, I, P, N, T e K.

Esse arquivo também possui a função `bcd_to_7seg`, usada para converter valores numéricos de 0 a 9 para o padrão de sete segmentos. Essa função é utilizada principalmente no estado `ST_LOCK`, para mostrar o tempo restante do bloqueio nos displays `HEX1` e `HEX0`.

8 Código VHDL

O projeto foi organizado em dois arquivos VHDL principais. O arquivo `cofre_pkg.vhd`, apresentado no Apêndice A, contém o pacote auxiliar com as constantes e funções usadas nos displays de sete segmentos. O arquivo `projeto.vhd`, apresentado no Apêndice B, contém a implementação principal do cofre digital, incluindo a entidade do projeto, os sinais internos, a máquina de estados, os contadores, a leitura das entradas e o controle dos LEDs e displays.

No arquivo principal, a senha é recebida por meio das chaves SW[8:0]. Cada posição da senha corresponde a uma combinação binária de nove chaves, permitindo valores entre 0 e 511. O usuário confirma quatro combinações sucessivas usando o botão KEY0; após a quarta confirmação, o sistema compara a senha digitada com a senha armazenada.

Na redefinição de senha, foi utilizado um cuidado adicional para evitar gravação parcial. Durante o estado ST_SET, os novos valores são armazenados primeiro em `senha_digitada`, funcionando como uma senha temporária. A senha definitiva, armazenada em `senha_salva`, só é atualizada depois que as quatro posições da nova senha são confirmadas. Assim, se o usuário pressionar KEY1 para cancelar a redefinição antes de terminar, a senha antiga permanece válida.

O código também mantém a chave SW9 como reset geral do sistema. Quando essa chave é ativada, o cofre retorna ao estado fechado, a senha padrão é restaurada, o contador de erros é zerado e o tempo de bloqueio é reiniciado. Os códigos completos utilizados no projeto estão apresentados nos Apêndices A e B.

9 Atribuição de Pinos

A atribuição de pinos foi feita conforme os recursos físicos da placa DE10-Lite. A Tabela 4 apresenta o mapeamento utilizado no Pin Planner do Quartus Prime, relacionando cada sinal declarado na entidade VHDL com o respectivo pino físico da placa. O clock de 50 MHz foi associado ao pino PIN_P11; os botões KEY0 e KEY1 foram associados a PIN_B8 e PIN_A7; as chaves SW0 a SW9, os LEDs e os displays de sete segmentos seguem a pinagem indicada no manual da DE10-Lite [2].

Tabela 4: Tabela de atribuição de pinos utilizada no Pin Planner.

Sinal VHDL	Pino físico	Função no projeto
Clock		
CLOCK_50	PIN_P11	Clock principal de 50 MHz da placa.
Botões		
KEY[0]	PIN_B8	Confirma a posição atual da senha; também fecha o cofre quando está em OPEN.
KEY[1]	PIN_A7	Cancela a digitação atual; quando o cofre está aberto, entra no modo de redefinição de senha.
Chaves deslizantes		
SW[0]	PIN_C10	Entrada de senha, bit 0.
SW[1]	PIN_C11	Entrada de senha, bit 1.
SW[2]	PIN_D12	Entrada de senha, bit 2.
SW[3]	PIN_C12	Entrada de senha, bit 3.
SW[4]	PIN_A12	Entrada de senha, bit 4.
SW[5]	PIN_B12	Entrada de senha, bit 5.
SW[6]	PIN_A13	Entrada de senha, bit 6.
SW[7]	PIN_A14	Entrada de senha, bit 7.
SW[8]	PIN_B14	Entrada de senha, bit 8.
SW[9]	PIN_F15	Reset geral do sistema, mantendo o funcionamento do código original.

Continua na próxima página.

Sinal VHDL	no	Pino físico	Função no projeto
LEDs			
LEDR[0]		PIN_A8	Indica cofre aberto.
LEDR[1]		PIN_A9	Indica cofre fechado ou em processo de verificação.
LEDR[2]		PIN_A10	Indica erro de senha.
LEDR[3]		PIN_B10	Indica modo de redefinição de senha.
LEDR[4]		PIN_D13	Indica erro acumulado.
LEDR[5]		PIN_C13	Indica erro acumulado.
LEDR[6]		PIN_E14	Indica erro acumulado.
LEDR[7]		PIN_D14	Indica erro acumulado.
LEDR[8]		PIN_A11	Indica erro acumulado.
LEDR[9]		PIN_B11	Pisca junto com os demais LEDs no estado LOCK.
Displays de sete segmentos			
HEX0[6]		PIN_C14	Segmento a do display HEX0.
HEX0[5]		PIN_E15	Segmento b do display HEX0.
HEX0[4]		PIN_C15	Segmento c do display HEX0.
HEX0[3]		PIN_C16	Segmento d do display HEX0.
HEX0[2]		PIN_E16	Segmento e do display HEX0.
HEX0[1]		PIN_D17	Segmento f do display HEX0.
HEX0[0]		PIN_C17	Segmento g do display HEX0.
HEX1[6]		PIN_C18	Segmento a do display HEX1.
HEX1[5]		PIN_D18	Segmento b do display HEX1.
HEX1[4]		PIN_E18	Segmento c do display HEX1.
HEX1[3]		PIN_B16	Segmento d do display HEX1.
HEX1[2]		PIN_A17	Segmento e do display HEX1.
HEX1[1]		PIN_A18	Segmento f do display HEX1.
HEX1[0]		PIN_B17	Segmento g do display HEX1.
HEX2[6]		PIN_B20	Segmento a do display HEX2.
HEX2[5]		PIN_A20	Segmento b do display HEX2.
HEX2[4]		PIN_B19	Segmento c do display HEX2.
HEX2[3]		PIN_A21	Segmento d do display HEX2.
HEX2[2]		PIN_B21	Segmento e do display HEX2.
HEX2[1]		PIN_C22	Segmento f do display HEX2.
HEX2[0]		PIN_B22	Segmento g do display HEX2.
HEX3[6]		PIN_F21	Segmento a do display HEX3.
HEX3[5]		PIN_E22	Segmento b do display HEX3.
HEX3[4]		PIN_E21	Segmento c do display HEX3.
HEX3[3]		PIN_C19	Segmento d do display HEX3.
HEX3[2]		PIN_C20	Segmento e do display HEX3.
HEX3[1]		PIN_D19	Segmento f do display HEX3.
HEX3[0]		PIN_E17	Segmento g do display HEX3.
HEX4[6]		PIN_F18	Segmento a do display HEX4.
HEX4[5]		PIN_E20	Segmento b do display HEX4.
HEX4[4]		PIN_E19	Segmento c do display HEX4.
HEX4[3]		PIN_J18	Segmento d do display HEX4.
HEX4[2]		PIN_H19	Segmento e do display HEX4.
HEX4[1]		PIN_F19	Segmento f do display HEX4.

Continua na próxima página.

Sinal VHDL	no	Pino físico	Função no projeto
HEX4[0]		PIN_F20	Segmento g do display HEX4.
HEX5[6]		PIN_J20	Segmento a do display HEX5.
HEX5[5]		PIN_K20	Segmento b do display HEX5.
HEX5[4]		PIN_L18	Segmento c do display HEX5.
HEX5[3]		PIN_N18	Segmento d do display HEX5.
HEX5[2]		PIN_M20	Segmento e do display HEX5.
HEX5[1]		PIN_N19	Segmento f do display HEX5.
HEX5[0]		PIN_N20	Segmento g do display HEX5.

10 Simulação e Testes

Para validar o funcionamento do sistema, foram considerados testes por simulação e também testes práticos diretamente na placa DE10-Lite. A simulação foi utilizada para verificar a lógica principal da máquina de estados antes da gravação na FPGA, observando se as transições ocorrem de acordo com as entradas aplicadas nos sinais SW e KEY.

Na simulação, os principais comportamentos analisados foram:

- inicialização do sistema no estado ST_CLOSE;
- entrada da senha correta e transição para o estado ST_OPEN;
- entrada de senha incorreta e exibição do estado ST_FAIL;
- incremento do contador de erros após tentativas incorretas;
- entrada no estado ST_LOCK após cinco erros consecutivos;
- decremento do contador de bloqueio durante o tempo de espera;
- retorno automático para o estado ST_CLOSE após o fim do bloqueio;
- entrada no modo ST_SET e redefinição da senha quando o cofre está aberto.

Esses testes permitiram confirmar que a máquina de estados responde corretamente aos comandos principais do usuário. Também foi possível verificar que o sistema ignora novas tentativas durante o estado de bloqueio e que o contador de erros é zerado após o desbloqueio automático.

Na placa, o teste prático deve seguir a sequência apresentada na Tabela 5.

Tabela 5: Sequência de testes sugerida para validação na placa.

Teste	Procedimento	Resultado esperado
Inicialização	Ligar ou resetar a placa.	Display em CLOSE.
Senha correta	Inserir a sequência inicial (1,2,3,4) usando SW[8:0] e KEY0.	Display em OPEN.
Senha incorreta	Inserir uma sequência diferente da senha salva.	Display em FAIL.
Cinco erros	Repetir cinco tentativas incorretas.	Display em LOCK e LEDs piscando.
Fim do bloqueio	Aguardar o tempo definido em LOCK_TIME.	Retorno para CLOSE.
Redefinição	Abrir o cofre, pressionar KEY1, digitar nova senha e confirmar as quatro posições com KEY0.	Nova senha salva e retorno para OPEN.
Cancelamento da redefinição	Entrar em SET, confirmar apenas parte da nova senha e pressionar KEY1.	Retorno para OPEN sem alterar a senha anterior.

As Figuras 2, 3 e 4 reservam espaço para as fotos da placa durante os principais estados de funcionamento.

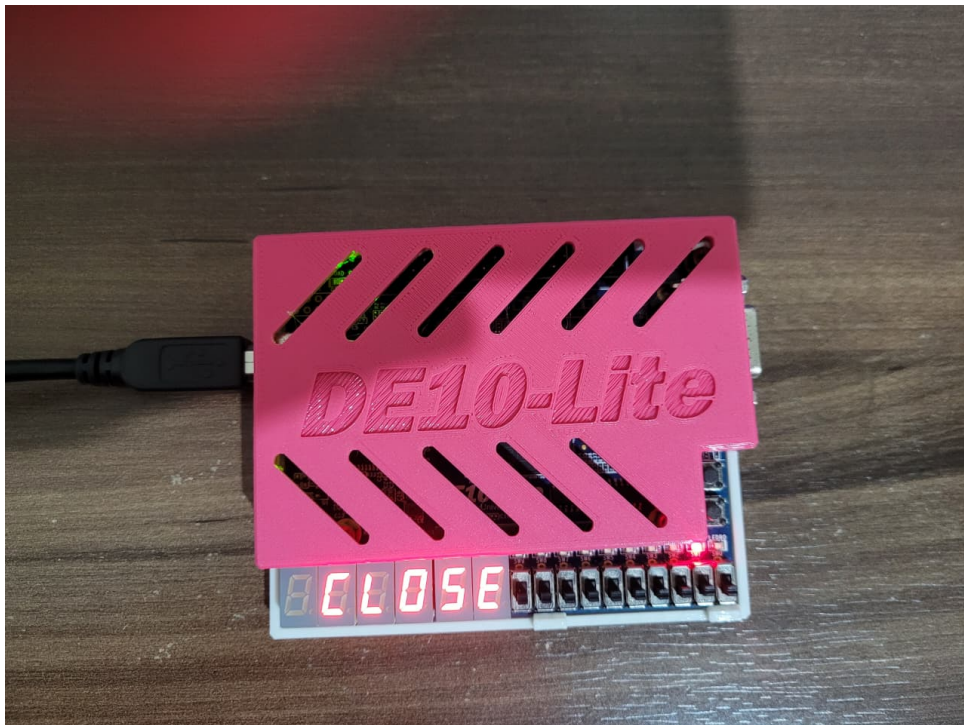


Figura 2: Placa DE10-Lite exibindo o estado fechado do cofre.

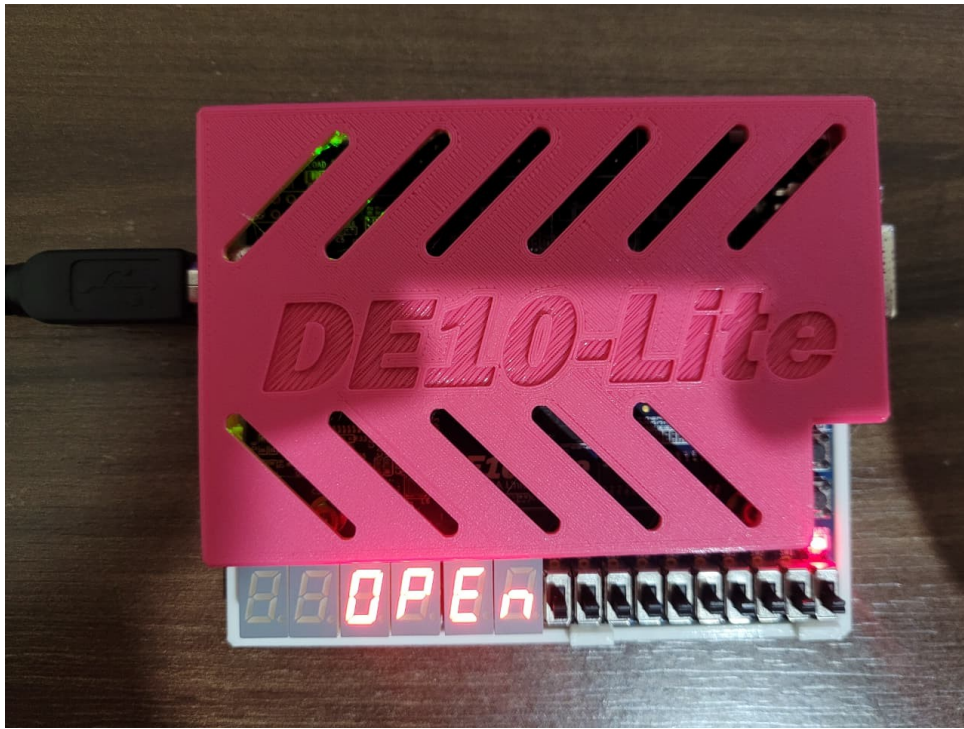


Figura 3: Placa DE10-Lite exibindo o estado aberto do cofre.

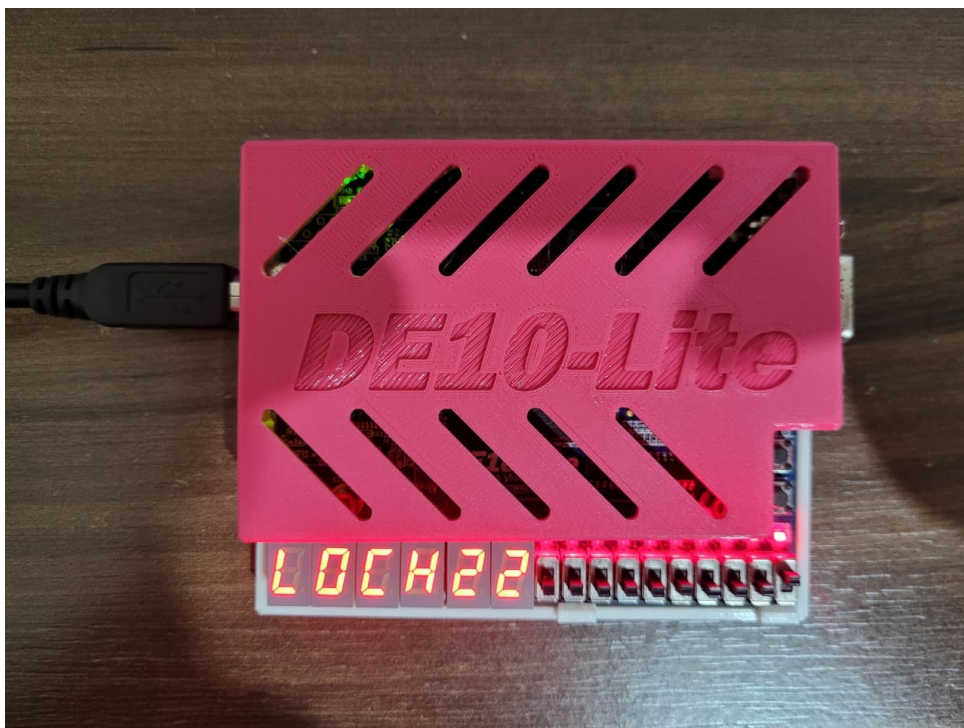


Figura 4: Placa DE10-Lite exibindo o estado de bloqueio do cofre.

11 Diagrama RTL

Após a síntese do projeto no Quartus Prime, o diagrama RTL pode ser utilizado para visualizar a estrutura lógica gerada a partir do código VHDL. Espera-se observar blocos relacionados aos registradores de senha, à máquina de estados, ao contador de tempo, ao controle dos LEDs e aos decodificadores dos displays de sete segmentos.

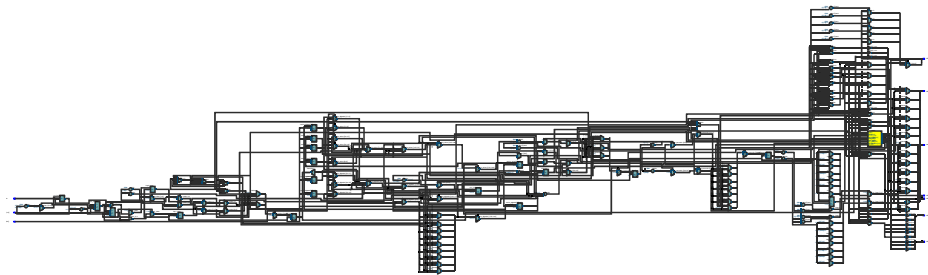


Figura 5: Diagrama RTL gerado para o projeto do cofre digital.

12 Guia de Utilização

Para utilizar o sistema na placa DE10-Lite, deve-se seguir o procedimento abaixo:

1. Manter **SW9** = 0 durante o uso normal, pois no código atual essa chave funciona como reset geral.
2. Ao ligar a placa, observar a mensagem **CLOSE** nos displays.
3. Para digitar a senha, ajustar a combinação desejada em **SW[8:0]**.
4. Pressionar **KEY0** para confirmar a posição atual da senha.
5. Repetir o processo até confirmar as quatro posições.
6. Se a senha estiver correta, o display mostrará **OPEN**.
7. Se a senha estiver incorreta, o display mostrará **FAIL** e uma tentativa será acumulada.
8. Após cinco erros, o sistema mostrará **LOCK**, os LEDs piscarão e o sistema aguardará o tempo definido em **LOCK_TIME** antes de liberar novas tentativas.

Para redefinir a senha, é necessário abrir o cofre primeiro. Com o display mostrando **OPEN**, pressiona-se **KEY1** para entrar em **SET**. Depois, basta inserir as quatro novas posições com **SW[8:0]** e confirmar cada uma com **KEY0**. Após a quarta confirmação, a nova senha é armazenada e o sistema retorna para **OPEN**. Caso **KEY1** seja pressionado antes de completar as quatro posições, a redefinição é cancelada e a senha anterior continua válida.

13 Conclusão

O projeto desenvolvido implementou um cofre digital funcional em VHDL, utilizando recursos básicos e intermediários da placa DE10-Lite. O sistema utiliza uma máquina de estados para organizar os modos de funcionamento, registradores para armazenar a senha, contadores para controlar o número de erros e o tempo de bloqueio, além de displays e LEDs para apresentar o estado do cofre ao usuário.

A implementação atende à proposta de um projeto final porque reúne vários conceitos trabalhados ao longo da disciplina, incluindo processos sequenciais, uso de *package*, funções, temporização com clock de 50 MHz, entrada por chaves, botões ativos em nível baixo e displays de sete segmentos. Como melhoria futura, o projeto poderia incluir um buzzer externo para alarme, senha mestre de recuperação ou confirmação dupla durante a redefinição de senha.

A Código do arquivo cofre_pkg.vhd

O Apêndice A apresenta o arquivo `cofre_pkg.vhd`, utilizado como pacote auxiliar do projeto. Esse arquivo concentra as constantes relacionadas aos padrões dos displays de sete segmentos, facilitando a organização do código principal.

Como os displays da DE10-Lite são acionados em lógica ativa baixa, cada letra ou número é representado por uma combinação de sete bits, em que o nível lógico baixo acende o segmento correspondente. Por esse motivo, o pacote define constantes como `SEG_C`, `SEG_L`, `SEG_O`, `SEG_S`, `SEG_E`, `SEG_F`, `SEG_A`, `SEG_I`, `SEG_P`, `SEG_N`, `SEG_T` e `SEG_K`, usadas para formar mensagens como `CLOSE`, `OPEN`, `FAIL`, `SET` e `LOCK`.

Além das constantes, o pacote também possui as funções `bcd_to_7seg` e `digit_to_7seg`. Essas funções convertem valores numéricos para o formato dos displays de sete segmentos, sendo utilizadas principalmente para exibir o contador regressivo durante o estado de bloqueio.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 package cofre_pkg is
6
7     -- Padroes para display de 7 segmentos em logica ativa baixa.
8     -- Ordem utilizada: a b c d e f g.
9     constant SEG_BLANK : std_logic_vector(6 downto 0) := "1111111";
10    constant SEG_DASH  : std_logic_vector(6 downto 0) := "1111110";
11
12    constant SEG_A : std_logic_vector(6 downto 0) := "0001000";
13    constant SEG_C : std_logic_vector(6 downto 0) := "0110001";
14    constant SEG_E : std_logic_vector(6 downto 0) := "0110000";
15    constant SEG_F : std_logic_vector(6 downto 0) := "0111000";
16    constant SEG_I : std_logic_vector(6 downto 0) := "1001111";
17    constant SEG_K : std_logic_vector(6 downto 0) := "1001000"; --
18        aproximacao visual
19    constant SEG_L : std_logic_vector(6 downto 0) := "1110001";
20    constant SEG_N : std_logic_vector(6 downto 0) := "1101010"; -- n
21        minusculo
22    constant SEG_O : std_logic_vector(6 downto 0) := "0000001";
23    constant SEG_P : std_logic_vector(6 downto 0) := "0011000";
24    constant SEG_S : std_logic_vector(6 downto 0) := "0100100";
25    constant SEG_T : std_logic_vector(6 downto 0) := "1110000"; -- t
26        minusculo
27
28    function bcd_to_7seg(bcd : std_logic_vector(3 downto 0)) return
29        std_logic_vector;
30    function digit_to_7seg(digit : integer) return std_logic_vector;
31
32 end package cofre_pkg;
33
34 package body cofre_pkg is
35
36     function bcd_to_7seg(bcd : std_logic_vector(3 downto 0)) return
37         std_logic_vector is
38     begin
39         case bcd is
40             when "0000" => return "0000001"; -- 0
41             when "0001" => return "1001111"; -- 1
42             when "0010" => return "0010010"; -- 2
43             when "0011" => return "0000110"; -- 3
44             when "0100" => return "1001100"; -- 4
45             when "0101" => return "0100100"; -- 5
```

```

41         when "0110" => return "0100000"; -- 6
42         when "0111" => return "0001111"; -- 7
43         when "1000" => return "0000000"; -- 8
44         when "1001" => return "0000100"; -- 9
45         when others => return SEG_E;      -- erro
46     end case;
47 end function bcd_to_7seg;
48
49 function digit_to_7seg(digit : integer) return std_logic_vector is
50 begin
51     if digit >= 0 and digit <= 9 then
52         return bcd_to_7seg(std_logic_vector(to_unsigned(digit, 4)));
53     else
54         return SEG_E;
55     end if;
56 end function digit_to_7seg;
57
58 end package body cofre_pkg;

```

Listing 1: Código completo do arquivo cofre_pkg.vhd.

B Código do arquivo projeto.vhd

O Apêndice B apresenta o arquivo principal `projeto.vhd`, responsável pela implementação completa do cofre digital. Nesse arquivo são declaradas as entradas e saídas físicas da placa DE10-Lite, incluindo o clock de 50 MHz, os botões, as chaves, os LEDs e os seis displays de sete segmentos.

A lógica principal foi construída por meio de uma máquina de estados finitos, composta pelos estados `ST_CLOSE`, `ST_INPUT`, `ST_VERIFY`, `ST_OPEN`, `ST_SET`, `ST_FAIL` e `ST_LOCK`. Cada estado representa uma etapa do funcionamento do cofre, como aguardar a senha, receber as combinações das chaves, verificar a senha, indicar abertura, redefinir a senha, sinalizar erro ou bloquear o sistema.

O código também possui um processo de temporização, responsável por gerar um pulso de 1 Hz a partir do clock de 50 MHz. Esse pulso é utilizado para controlar o tempo de exibição do estado de falha, o contador regressivo durante o bloqueio e o piscar dos LEDs no estado `ST_LOCK`.

A senha é armazenada em um vetor com quatro posições, sendo que cada posição recebe uma combinação de `SW[8:0]`. Assim, ao pressionar `KEY0`, o valor atual das chaves é salvo como uma posição da senha digitada. Após quatro confirmações, o sistema compara a senha digitada com a senha armazenada e realiza a transição para `OPEN`, `FAIL` ou `LOCK`, conforme o resultado.

Na redefinição, o código usa o vetor `senha_digitada` como armazenamento temporário. Isso evita que uma nova senha incompleta seja gravada. Se o usuário cancelar a redefinição com `KEY1` antes da quarta confirmação, o sistema retorna para `ST_OPEN` sem alterar `senha_salva`. A senha definitiva só é modificada quando as quatro posições são confirmadas.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 use work.cofre_pkg.all;
5
6 entity projeto is
7     generic (
8         CLK_FREQ      : integer := 50000000; -- clock da DE10-Lite: 50 MHz
9         LOCK_TIME     : integer := 30;       -- tempo de bloqueio em
10            segundos
11         KEY_SAMPLE_HZ : integer := 20        -- amostragem dos botoes: 20
12            Hz
13     );
14     port (
15         CLOCK_50 : in  std_logic;
16         KEY      : in  std_logic_vector(1 downto 0); -- botoes ativos em
17            nivel baixo
18         SW       : in  std_logic_vector(9 downto 0);
19         LEDR     : out std_logic_vector(9 downto 0);
20         HEX0     : out std_logic_vector(6 downto 0);
21         HEX1     : out std_logic_vector(6 downto 0);
22         HEX2     : out std_logic_vector(6 downto 0);
23         HEX3     : out std_logic_vector(6 downto 0);
24         HEX4     : out std_logic_vector(6 downto 0);
25         HEX5     : out std_logic_vector(6 downto 0);
26     );
27 end entity projeto;
28
29 architecture projeto of projeto is
30     type estado_type is (
31         ST_CLOSE,
32         ST_INPUT,
```

```

31     ST_VERIFY,
32     ST_OPEN,
33     ST_SET,
34     ST_FAIL,
35     ST_LOCK
36 );
37
38 -- Cada posicao da senha agora pode usar uma combinacao de SW0 ate SW8
39 -- SW[8:0] gera valores de 0 ate 511.
40 type senha_type is array (0 to 3) of integer range 0 to 511;
41
42 constant KEY_DIV : integer := CLK_FREQ / KEY_SAMPLE_HZ;
43
44 signal estado      : estado_type := ST_CLOSE;
45
46 -- Senha inicial continua sendo (1, 2, 3, 4), mas agora cada numero
47 -- representa uma combinacao binaria em SW[8:0].
48 signal senha_salva : senha_type := (1, 2, 3, 4);
49 signal senha_digitada : senha_type := (0, 0, 0, 0);
50
51 signal digit_index : integer range 0 to 3 := 0;
52 signal erros       : integer range 0 to 5 := 0;
53 signal lock_seconds : integer range 0 to 99 := LOCK_TIME;
54
55 signal tick_1hz : std_logic := '0';
56 signal blink    : std_logic := '0';
57 signal key0_press : std_logic := '0';
58 signal key1_press : std_logic := '0';
59 signal key0_sample : std_logic := '1';
60 signal key1_sample : std_logic := '1';
61
62 -- Agora o valor digitado vem de SW0 ate SW8.
63 signal digito_atual : integer range 0 to 511;
64 signal digito_valido : std_logic;
65
66 begin
67
68     --
69     -----
70
71     -- Entrada da senha.
72     -- Antes: SW(3 downto 0), valores de 0 a 15.
73     -- Agora: SW(8 downto 0), valores de 0 a 511.
74     --
75     -- SW9 continua separado, funcionando como reset geral, igual ao
76     -- original.
77     --
78     -----
79
80     digito_atual <= to_integer(unsigned(SW(8 downto 0)));
81     digito_valido <= '1';
82
83     --
84     -----
85
86     -- Divisao de clock para 1 Hz e leitura mais lenta dos botoes.
87     -- A leitura em 20 Hz reduz o efeito de repiques mecanicos simples.
88     --
89     -----

```

```

82 process(CLOCK_50)
83     variable cnt_1hz : integer range 0 to CLK_FREQ - 1 := 0;
84     variable cnt_key : integer range 0 to KEY_DIV - 1 := 0;
85 begin
86     if rising_edge(CLOCK_50) then
87         tick_1hz    <= '0';
88         key0_press  <= '0';
89         key1_press  <= '0';
90
91         if cnt_1hz = CLK_FREQ - 1 then
92             cnt_1hz := 0;
93             tick_1hz <= '1';
94             blink <= not blink;
95         else
96             cnt_1hz := cnt_1hz + 1;
97         end if;
98
99         if cnt_key = KEY_DIV - 1 then
100             cnt_key := 0;
101
102             if key0_sample = '1' and KEY(0) = '0' then
103                 key0_press <= '1';
104             end if;
105
106             if key1_sample = '1' and KEY(1) = '0' then
107                 key1_press <= '1';
108             end if;
109
110             key0_sample <= KEY(0);
111             key1_sample <= KEY(1);
112         else
113             cnt_key := cnt_key + 1;
114         end if;
115     end if;
116 end process;
117
118 --
119 -- -----
120 -- Maquina de estados principal do cofre.
121 -- SW(9) funciona como reset geral: retorna para CLOSE e senha inicial
122 --
123 -- -----
124
125 process(CLOCK_50)
126 begin
127     if rising_edge(CLOCK_50) then
128
129         if SW(9) = '1' then
130             estado          <= ST_CLOSE;
131             senha_salva     <= (1, 2, 3, 4);
132             senha_digitada  <= (0, 0, 0, 0);
133             digit_index     <= 0;
134             erros           <= 0;
135             lock_seconds    <= LOCK_TIME;
136
137         else
138             case estado is

```

```

136
137     when ST_CLOSE =>
138         digit_index <= 0;
139
140         if key0_press = '1' and digito_valido = '1' then
141             senha_digitada(0) <= digito_atual;
142             digit_index <= 1;
143             estado <= ST_INPUT;
144         end if;
145
146     when ST_INPUT =>
147         if key1_press = '1' then
148             digit_index <= 0;
149             estado <= ST_CLOSE;
150
151         elsif key0_press = '1' and digito_valido = '1'
152             then
153             senha_digitada(digit_index) <= digito_atual;
154
155             if digit_index = 3 then
156                 digit_index <= 0;
157                 estado <= ST_VERIFY;
158             else
159                 digit_index <= digit_index + 1;
160             end if;
161         end if;
162
163     when ST_VERIFY =>
164         if senha_digitada = senha_salva then
165             erros <= 0;
166             estado <= ST_OPEN;
167         else
168             if erros = 4 then
169                 erros <= 5;
170                 lock_seconds <= LOCK_TIME;
171                 estado <= ST_LOCK;
172             else
173                 erros <= erros + 1;
174                 estado <= ST_FAIL;
175             end if;
176         end if;
177
178     when ST_FAIL =>
179         if tick_1hz = '1' then
180             estado <= ST_CLOSE;
181         end if;
182
183     when ST_OPEN =>
184         -- KEY0 fecha o cofre. KEY1 entra no modo de
185         redefinicao.
186         if key0_press = '1' then
187             estado <= ST_CLOSE;
188         elsif key1_press = '1' then
189             digit_index <= 0;
190             estado <= ST_SET;
191         end if;
192
193     when ST_SET =>
194         -- KEY1 cancela a redefinicao e volta para OPEN.
195         if key1_press = '1' then

```

```

194         digit_index <= 0;
195         estado <= ST_OPEN;
196
197     elsif key0_press = '1' and digito_valido = '1'
198     then
199         senha_digitada(digit_index) <= digito_atual;
200
201         if digit_index = 3 then
202             senha_salva(0) <= senha_digitada
203             (0);
204             senha_salva(1) <= senha_digitada(1);
205             senha_salva(2) <= senha_digitada(2);
206             senha_salva(3) <= digito_atual;
207
208             digit_index <= 0;
209             erros <= 0;
210             estado <= ST_OPEN;
211         else
212             digit_index <= digit_index + 1;
213         end if;
214     end if;
215
216 when ST_LOCK =>
217     -- Durante o bloqueio, as tentativas sao ignoradas
218     .
219     if tick_1hz = '1' then
220         if lock_seconds <= 1 then
221             lock_seconds <= 0;
222             erros <= 0;
223             estado <= ST_CLOSE;
224         else
225             lock_seconds <= lock_seconds - 1;
226         end if;
227     end if;
228
229 end case;
230 end if;
231 end if;
232 end process;
233
234 --
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

247         leds(2) := '1'; -- erro
248
249         when ST_LOCK =>
250             leds := (others => blink); -- bloqueado: LEDs piscando
251
252         when ST_SET =>
253             leds(3) := '1'; -- modo de redefinicao
254     end case;
255
256     if estado /= ST_LOCK then
257         for i in 0 to 4 loop
258             if erros > i then
259                 leds(4 + i) := '1';
260             end if;
261         end loop;
262     end if;
263
264     LEDR <= leds;
265 end process;
266
267 --
268
269 -----
270
271 -- Controle dos displays HEX.
272 --
273 -----
274
275 process(estado, lock_seconds)
276 begin
277     HEX5 <= SEG_BLANK;
278     HEX4 <= SEG_BLANK;
279     HEX3 <= SEG_BLANK;
280     HEX2 <= SEG_BLANK;
281     HEX1 <= SEG_BLANK;
282     HEX0 <= SEG_BLANK;
283
284     case estado is
285         when ST_CLOSE =>
286             -- CLOSE
287             HEX4 <= SEG_C;
288             HEX3 <= SEG_L;
289             HEX2 <= SEG_0;
290             HEX1 <= SEG_S;
291             HEX0 <= SEG_E;
292
293             when ST_INPUT | ST_VERIFY =>
294                 -- PASS
295                 HEX3 <= SEG_P;
296                 HEX2 <= SEG_A;
297                 HEX1 <= SEG_S;
298                 HEX0 <= SEG_S;
299
300             when ST_OPEN =>
301                 -- OPEN
302                 HEX3 <= SEG_0;
303                 HEX2 <= SEG_P;
304                 HEX1 <= SEG_E;
305                 HEX0 <= SEG_N;
306
307             when ST_SET =>

```

```

303         -- SET
304         HEX2 <= SEG_S;
305         HEX1 <= SEG_E;
306         HEX0 <= SEG_T;
307
308     when ST_FAIL =>
309         -- FAIL
310         HEX3 <= SEG_F;
311         HEX2 <= SEG_A;
312         HEX1 <= SEG_I;
313         HEX0 <= SEG_L;
314
315     when ST_LOCK =>
316         -- LOCK + contador de segundos
317         HEX5 <= SEG_L;
318         HEX4 <= SEG_0;
319         HEX3 <= SEG_C;
320         HEX2 <= SEG_K;
321         HEX1 <= digit_to_7seg(lock_seconds / 10);
322         HEX0 <= digit_to_7seg(lock_seconds mod 10);
323     end case;
324 end process;
325
326 end architecture projeto;

```

Listing 2: Código completo do arquivo principal `projeto.vhd`.

Referências

- [1] Atividade 9: Projeto final. Documento de enunciado fornecido pelo professor da disciplina de Lógica Reconfigurável, UTFPR Campus Apucarana, 2026.
- [2] TERASIC. *DE10-Lite User Manual*. Versão de 5 de junho de 2020.
- [3] INTEL. *Quartus Prime Lite Edition*. Ambiente utilizado para síntese, simulação, configuração de pinos e programação da FPGA.